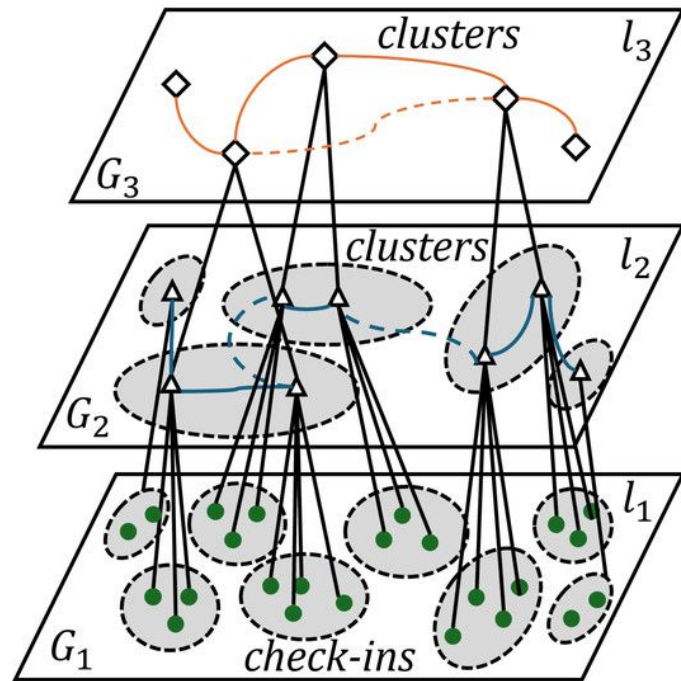


HiRAG

Retrieval-Augmented
Generation with
Hierarchical Knowledge

Paul Béglin
Ekaterina Bogush
Zahra Siddique



Plan



1

Concept de HiRAG

2

Problèmes de HiRAG

3

Améliorations implémentées

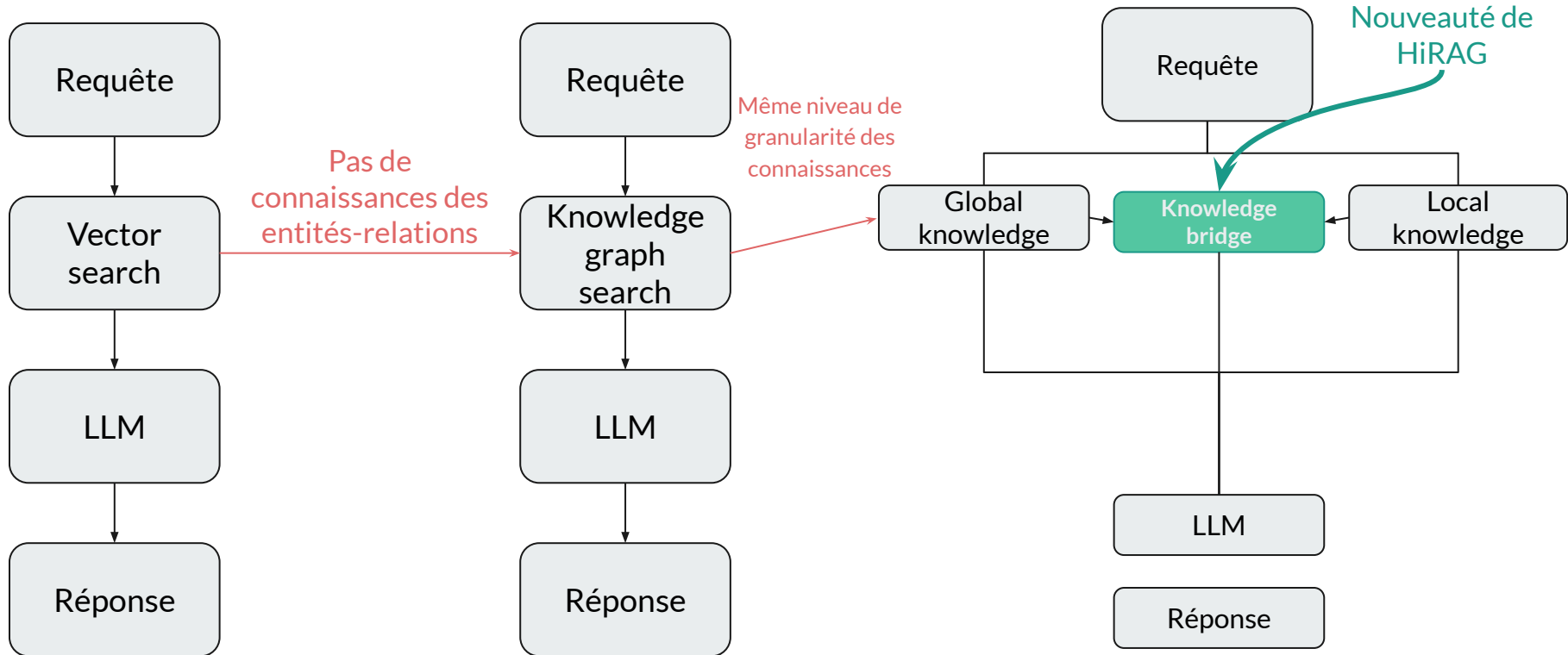
4

Évaluation

5

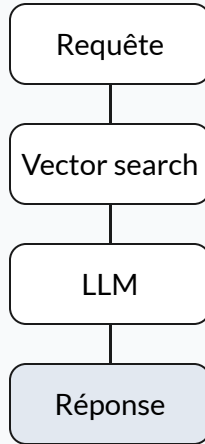
Conclusion

Concept de HiRAG. Comment améliorer le RAG?



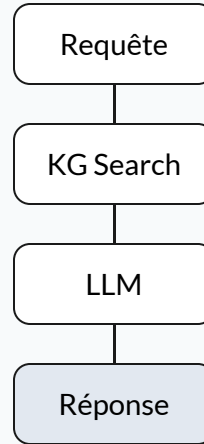
Concept de HiRAG : Améliorer le RAG

Standard RAG



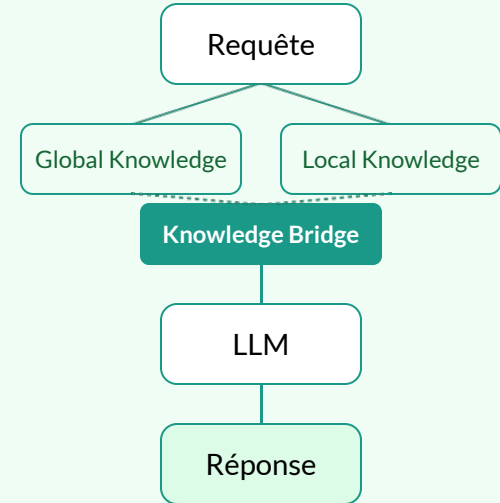
Pas de connaissances des entités-relations

Graph RAG



Même granularité (difficile pour les requêtes larges)

HiRAG (Hierarchical)



Nouveauté de HiRAG

Problèmes de HiRAG

1

Les chunks pertinents **entiers** sont passés à LLM



Non mentionné dans l'article comme faisant partie de l'implémentation

After retrieving the three-level hierarchical knowledge, i.e., local-level descriptions of the individual entities in $\hat{\mathcal{V}}$, global-level community reports of the communities in $\hat{\mathcal{P}}$, and bridge-level descriptions of the triples in $\hat{\mathcal{R}}$, we feed them as the context to the LLM to generate a comprehensive answer to the query. We also provide the detailed procedures of HiRAG with pseudocodes in Appendix C.

2

Trouver des entités locales pertinentes avec la **recherche dense** seulement, pas de reranker

3

Bridge entre les entités locales et les communautés globales n'est pas query-aware

Saturating LLM context

Solution : séparer le chunk en phrases/fenêtres (selon la punctuation and `\n`), calculer le score pour chaque phrase, choisir une fenêtre par chunk

Stratégie de chevauchement avec la requête

$$\text{overlap}(Q, W) = |Q \cap T(W)|$$

$$\text{density}(Q, W) = \frac{|Q \cap T(W)|}{\max(1, |T(W)|)}$$

$$\text{score}(W) = \left(\text{overlap}(Q, W), \text{density}(Q, W), -|W| \right)$$

$$W^* = \arg \max_W \text{score}(W)$$

Q = ensemble des tokens normalisés de la requête

$T(W)$ = fenêtre candidate de texte

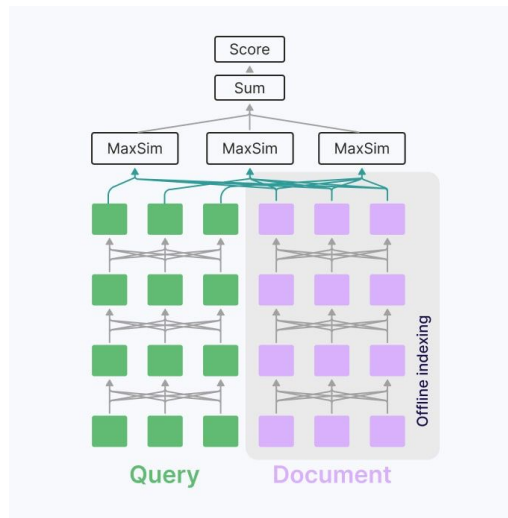
W = set of normalized tokens in window

Normalisation = mise en minuscules, tokenization, suppression des tokens d'un seul caractère, suppression des stopwords

Reranker. Late-interaction mechanism ColBERT

Solution : recherche dense sur les entités locales (limiter à top-100), ensuite applique le reranker ColBERT, prendre top-20

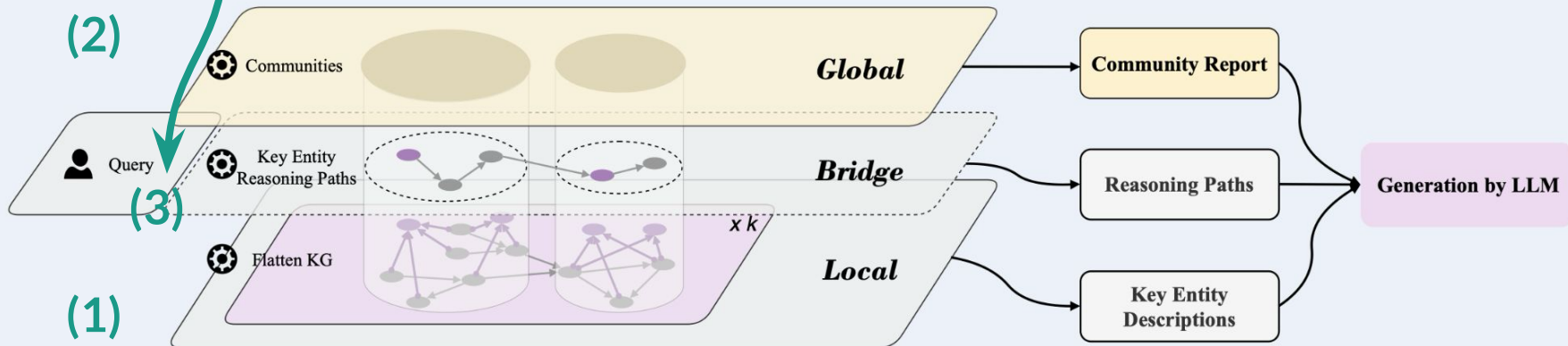
Model : [answerdotai/answerai-colbert-small-v1](#)



Condition reasoning path by the query

Idea of HiRAG:
reasoning path = shortest
path between
communities-aware
ordering of local entities

Condition reasoning path by
the query



HiRetrieval: Retrieval with Hierarchical Knowledge

Condition reasoning path by the query

1

Dijkstra pondéré : Parmi les chemins **complets**, lequel a le coût total minimal ?

2

Minimax : Parmi les chemins **complets**, lequel a le meilleur 'worst edge' ?

2.1

Minimax budgeted : Parmi tous les **petits** chemins **complets**, lequel a la meilleur pire arête?

3

Monte Carlo Tree Search : En construisant un bridge, quel est le meilleur choix garantissant une récompense long terme ?

Coût d'une arête

1 Embedding d'une arête

Calculer embedding sur

```
source: <source entity name>
<source entity description>
relation: <relation description>
target: <target entity name>
<target entity description>
```

2 Définir le coût d'une arête $e = (u, v)$ par rapport à la requête q

$$\text{similarity_score}(q, e) = \cos(\mathbf{q}, \mathbf{e})$$

$$\text{bounded_distance}(q, e) = \max(0, 1 - \text{similarity_score}(q, e))$$

$$\text{minimize } c(e \mid q) = \text{bounded_distance}(q, e) + 0.15 \cdot \text{hub_penalty}(e) + 0.05 \cdot \text{inverse_weight_penalty}(e)$$

$$\text{maximize } u(e \mid q) = \max(0, 1 - c(e \mid q)) \quad \text{utilité (utilisée en MCTS)}$$

Pénaliser des noeuds génériques

Pénaliser les relations faibles plus que les fortes

Relationship strength (définie pendant la construction du graphe)

$$\text{hub_penalty}(e) = \frac{\deg(u) + \deg(v)}{\max_{(i,j) \in E} (\deg(i) + \deg(j))}$$

$$\text{inverse_weight_penalty}(e) = \frac{1}{1 + w_e}$$

Coût d'une arête

1 Embedding d'une arête

```
source: <source entity name>
<source entity description>
relation: <relation description>
target: <target entity name>
```

2 Coût d'une arête $e = (u, v)$

Calculé par rapport à la requête q

Objectif : Équilibrer distance, centralité et force des liens.

$$\text{similarity_score}(q, e) = \cos(\mathbf{q}, \mathbf{e}) \quad \text{bounded_distance}(q, e) = \max(0, 1 - \text{similarity_score}(q, e))$$

$$c(e \mid q) = \text{bounded_distance}(q, e) + 0.15 \cdot \text{hub_penalty}(e) + 0.05 \cdot \text{inverse_weight_penalty}(e) \quad \text{minimize cost}$$

$$u(e \mid q) = \max(0, 1 - c(e \mid q)) \quad \text{maximize utility}$$

utilisée en MCTS

Hub Penalty

Pénalise les noeuds trop génériques pour favoriser la spécificité des relations.

$$\text{hub_penalty}(e) = \frac{\log(d(u)) + \log(d(v))}{\log(d(u)) \cdot \log(d(v))}$$

Weight Penalty

Pénalise les relations faibles pour renforcer la cohérence.

$$\text{inverse_weight_penalty}(e) = \frac{1}{1 + u}$$

Utilité MCTS

Maximise l'intérêt informationnel local pour le raisonnement.

Dijkstra

Objective de HiRAG (par défaut) $\arg \min_P |P|$

$$P = (e_1, e_2, \dots, e_m)$$

Dijkstra pondéré $P^* = \arg \min_P \sum_{k=1}^m c(e_k \mid q)$

Limitation connue : minimise la somme des coûts, peut accepter une arête à coût conséquent si toutes les autres sont gratuites

Dijkstra

Objective de HiRAG (par défaut)

$$\arg \min_P |P| \quad P = (e_1, e_2, \dots, e_m)$$

Dijkstra pondéré

$$P^* = \arg \min_P \sum_{k=1}^m c(e_k \mid q)$$



Limitation connue

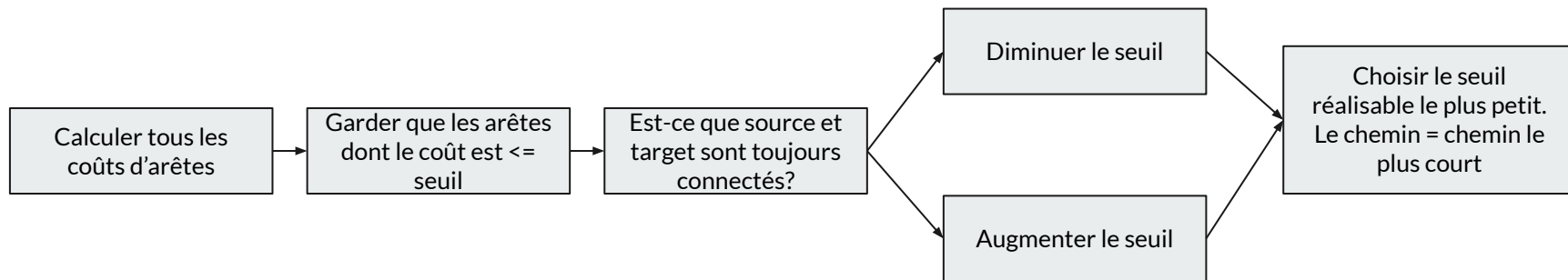
Minimise la somme des coûts globale. Cela peut conduire à l'acceptation d'une arête à **coût conséquent** si la majorité des autres arêtes du chemin sont gratuites.

Minimax

$$P = (e_1, e_2, \dots, e_m)$$

Objective de HiRAG (par défaut) $\arg \min_P |P|$

Version à seuil de minimax pour **simplifier l'implémentation à budget** de minimax



Minimax
classique

$$P^* = \arg \min_P \max_{e \in P} c(e \mid q)$$

Minimax avec le
budget

Budget local

$$P^* = \min_P \max_{e \in P} c(e \mid q) \quad \text{s.t.} \quad |P| - 1 \leq 120$$

Budget global

$$\sum_i (|P_i| - 1) \leq 220 \quad (v_1 \rightarrow v_2), (v_2 \rightarrow v_3), \dots, (v_{k-1} \rightarrow v_k)$$

Monte Carlo Tree Search (MCTS)

Trouver un chemin de liaison qui utilise des arêtes pertinentes, atteint la cible, reste court et ne consomme pas trop tokens.

1

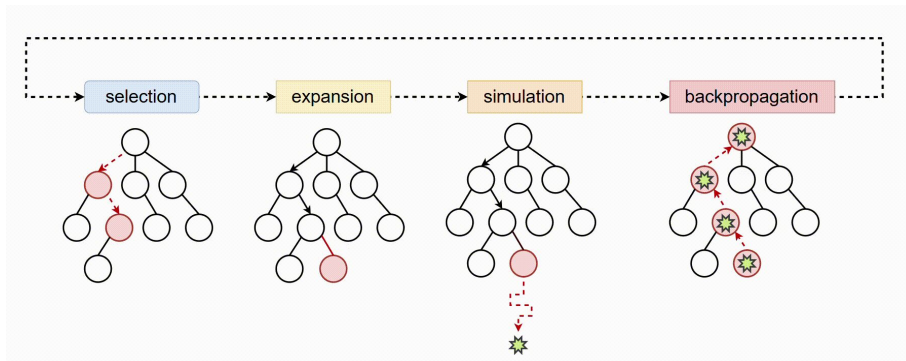
Calculer le coût des arêtes

2

Restreindre l'espace de recherche à un **sous graphe candidat** autour de la source et target.
Permet d'accélérer la convergence et limiter le branching factor

3

Appliquer MCTS



4

Décision : comparer à **Dijkstra pondéré**. Si le chemin trouvé par MCTS est mieux que Dijkstra, alors on arrête l'exploration (early stopping)

Construire le sous-graphe

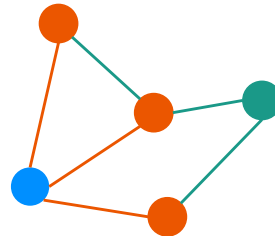
1. Restreindre le graphe aux voisins des nœuds source et cible situés dans un rayon de r sauts

$$V_{\text{cand}} = N_r(s) \cup N_r(t) \cup \{s, t\}$$

$$r \in \{2, 3, 4\}$$

↑
augmente si la source et la cible
deviennent déconnectées

$r = 1$



Pruning : Pour chaque nœud v , prendre que les voisins avec la meilleure utilité et les moins génériques

$$\mathcal{N}_{\text{keep}}(v) = \text{Top-12}_{x \in \mathcal{N}(v)} (u(v, x \mid q), -\deg(x))$$

Si le nœud source et cible devient déconnectées, éviter le pruning

Appliquer MCTS

3.0

Initialiser la racine de l'arbre d'exploration

noeuds visités à l'instant t

nombre de tokens estimés

noeud courant

chemin à l'instant t

source

$$s_0 = (v_0, P_0, V_0, \tau_0) = (s, [s], \{s\}, 0)$$

3.1

Sélection : choisir les enfants avec le meilleur UCT

nombre de fois l'état s a été visité

$$UCT(s, a) = \bar{Q}(s, a) + 1.2 \sqrt{\frac{\ln N(s)}{N(s, a)}}$$

Hypothèse souvent posée en MCTS

$$s \xrightarrow{a} s'$$

$$Q(s, a) = Q(s')$$

nombre de fois l'action a a été pris l'état s

Pour le noeud-enfant non-visité (pour garantir l'exploration)

$$UCT(s, a) = +\infty$$

3.3

Expansion : pour le noeud courant v

1. Ordonner les actions en ordre croissant : **max** l'utilité, **min** la distance, **min** le nombre de noeuds visités

$$\text{score}_{\text{nbr}}(x) = -u(v, x \mid q) - 0.4 \cdot \frac{1}{1 + d(x, t)} + 0.2 \cdot \mathbf{1}[x \in V]$$

↑
distance la plus courte dans le sous-graphe

2. Expand au plus $K(v)$ enfants

$$K(v) = \max \left(1, \left\lfloor 2 \cdot \sqrt{N(v)} \right\rfloor \right)$$

3.4

Simulation/rollout (profondeur maximale = 4)

1. Filtrer les noeuds non-visités $\mathcal{N}_{\text{rollout}}(v) = \{x \in \mathcal{N}(v) : x \notin V\}$

2. Choisir la meilleur action (epsilon-greedy)

$$a_t = \begin{cases} \text{best candidate} & \text{with prob. 0.9} \\ \text{random top-3 candidate in } \mathcal{N}_{\text{rollout}}(v) & \text{with prob. 0.1} \end{cases}$$

$$\text{terminal}(s) = \begin{cases} \text{target_reached} & v_t = \text{target} \\ \text{depth_limit} & |P_t| - 1 \geq 120 \\ \text{token_budget} & \tau_t \geq 12500 \\ \text{dead_end} & |\mathcal{A}(s)| = 0 \end{cases}$$

3.5

Backpropagation $P = (v_0, \dots, v_L)$

1. Évaluer la récompense : **max** l'utilité, bonus si le dernier état est l'état terminal, **pénaliser** la longueur du chemin et **pénaliser** le nombre de tokens

$$\text{mean_edge_utility}(P) = \frac{1}{L} \sum_{i=0}^{L-1} u(v_i, v_{i+1} \mid q) \qquad \text{token_ratio}(P) = \min \left(1, \frac{\sum_{e \in P} \text{tok}(e)}{T_{\max}} \right)$$

$$R(P) = \text{mean_edge_utility}(P) + \mathbf{1}[v_L = \text{target}] - 0.02 \cdot L - 0.15 \cdot \text{token_ratio}(P)$$

Pour chaque noeud sur le chemin s

$$N(s) \leftarrow N(s) + 1 \qquad \text{nombre de fois l'état } s \text{ a été visité}$$

$$W(s) \leftarrow W(s) + R(P) \qquad \text{la récompense cumulative}$$

$$\bar{Q}(s) = \frac{W(s)}{N(s)} \qquad \text{value function estimée de l'état } s$$

Évaluation



2 Datasets de UltraDomain Benchmark : Mix Agriculture

Modèle d'embeddings :
text-embedding-3-small

LLM (réponse gén):
OpenAI GPT 5.4 mini

LLMaJ : OpenAI GPT 5.4
mini

| Dataset | Docs | Questions | Chunks | Nodes | Edges | Community Reports | Entity
Vectors | Chunk Vectors |
|---|---:|---:|---:|---:|---:|---:|---:|
| Agriculture | `12` | `100` | `1756` | `23224` | `48428` | `4639` | `23224` | `1756` |
| Mix | `61` | `130` | `579` | `16356` | `25960` | `2695` | `16356` | `579` |

Évaluation. LLMaJ

LLMaJ strict : évaluer 2 fois en échangeant l'ordre des documents: compare response from classic HiRAG to our variant. Sample 30 questions per dataset

variant	valid_comparisons				wins	losses	ties	win_rate	loss_rate	tie_rate
mean_latency_seconds				mean_context_tokens				mean_bridge_edge_count		
hi_mcts	60	29	31	0	0.48	0.52	0.00	0.96	4573.07	12.40
hi_minmax	60	26	34	0	0.43	0.57	0.00	2.49	4576.73	12.33
hi_minmax_budgeted	60	23	37	0	0.38	0.62	0.00	2.43	4576.73	12.33
hi_rerank_weighted	60	25	35	0	0.42	0.58	0.00	5.01	4974.43	12.13
hi_weighted	60	24	36	0	0.40	0.60	0.00	1.19	4551.37	12.00
naive	60	40	20	0	0.67	0.33	0.00	0.41	6614.23	0.00

ulture

	valid_comparisons	wins	losses	ties	win_rate	loss_rate	tie_rate	mean_latency_seconds	mean_context_tokens	mean_bridge_edge_count
--	-------------------	------	--------	------	----------	-----------	----------	----------------------	---------------------	------------------------

Évaluation. LLMaJ

LLMaJ strict : évaluer 2 fois en échangeant l'ordre des documents: compare response from classic HiRAG to our variant. Sample 30 questions per dataset

Mix

variant	valid_comparisons				wins	losses	ties	win_rate	loss_rate	tie_rate
mean_latency_seconds		mean_context_tokens			mean_bridge_edge_count					
hi_mcts	60	29	31	0	0.48	0.52	0.00	0.96	4573.07	12.40
hi_minmax	60	26	34	0	0.43	0.57	0.00	2.49	4576.73	12.33
hi_minmax_budgeted	60	23	37	0	0.38	0.62	0.00	2.43	4576.73	12.33
hi_rerank_weighted	60	25	35	0	0.42	0.58	0.00	5.01	4974.43	12.13
hi_weighted	60	24	36	0	0.40	0.60	0.00	1.19	4551.37	12.00
naive	60	40	20	0	0.67	0.33	0.00	0.41	6614.23	0.00

Évaluation : Performance LLMaJ

LLMaJ Strict: Évaluation en 2 étapes avec inversion de l'ordre des documents. Comparaison entre HiRAG classique et variantes.
Échantillon de 30 questions par dataset.



MIX

Variant	Wins	Losses	Win Rate	Latency (s)	Tokens	Bridge Edges
---------	------	--------	----------	-------------	--------	--------------

hi_mcts	2931	48.3%	0.964	573	12.4	
hi_minmax	2634	43.3%	2.494	576	12.3	
hi_minmax_budgeted	2337	38.3%	2.434	576	12.3	
hi_rerank_weighted	2535	41.7%	5.014	974	12.1	
hi_weighted	2436	40.0%	1.194	551	12.0	
naive (Baseline)	4020	66.7%	0.416	614	0.0	

Évaluation : LLMaJ Metrics

LLMaJ strict : évaluer 2 fois (ordre inversé) | Comparaison HiRAG vs Variants | 30 questions/dataset

MIX

Variant	Wins	Losses	Win Rate	Latency (s)	Tokens	Bridge	Edges
---------	------	--------	----------	-------------	--------	--------	-------

hi_mcts	2931	148	3.3%	0.964	5,573	12.4	
hi_minmax	2634	443	3.3%	2.494	5,576	12.3	
hi_weighted	2436	40	0.0%	1.194	551	12.0	
naive	4020	66	6.7%	0.416	6,140	0	

AGRICULTURE

Variant	Wins	Losses	Win Rate	Latency (s)	Tokens	Bridge	Edges
---------	------	--------	----------	-------------	--------	--------	-------

hi_mcts	3227	53	3.3%	1.026	2,191	14.1	
hi_minmax	2732	45	0.0%	2.556	2,681	13.8	
hi_weighted	3326	55	0.0%	1.256	1,831	13.5	
naive	5378	8	3.3%	0.386	0,160	0	

Évaluation. LLMaJ

LLMaJ : compare response from classic HiRAG to our variant. Sample 30 questions per dataset

Variant	Wins	Losses	Win Rate	Latency (s)	Context Tokens
hi	-	-	-	0.22	5039
hi_mcts	29	31	48.3%	0.0	4573
hi_minmax	26	34	43.3%	0.0	4576
hi_weighted	24	36	40.0%	0.0	4551
naive	40	20	66.7%	0.0	6614

Variant	Wins	Losses	Win Rate	Latency (s)	Context Tokens
hi_mcts	32	27	53.3%	0.0	6219
hi_minmax	27	32	45.0%	0.0	6268
hi_weighted	33	26	55.0%	0.0	6183
naive	53	7	88.3%	0.0	6016

Évaluation. Human annotation

But : comparer HiRAG classique à nos 3 variantes :

Variante	Win rate	Densité du span utile	Gain moyen
HiRAG classique	-	28.7%	-
Naive	65.0%	32.0%	+0.30
Minmax budgeted	45.0%	32.1%	-0.10
MCTS	52.5%	30.4%	+0.05

Packet 1 | Query mix:11

How did The Cleveland Show come into existence, and what was its premise?

Answer A
Development
The Cleveland Show came into existence as a spin-off of *Family Guy*. Seth MacFarlane initially conceived the series in 2007 while he was working on *Family Guy* and *American Dad!* for Fox. The show was developed around Cleveland Brown, a character already established on *Family Guy*, and Fox picked up the project after it appeared on the network's development slate in early 2008.

Answer B
Creation and development
The Cleveland Show came into existence as a spin-off of *Family Guy*. According to the data, it was created by Seth MacFarlane, Richard Appel, and Mike Henry for Fox Broadcasting Company. The show was developed for Fox, and Fox ultimately aired it. The pilot was pitched to Fox, leaked online in June 2009, and

Your annotation

Does Answer A answer the question?
☒ yes ☐ partially ☐ no

Does Answer B answer the question?
☒ yes ☐ partially ☐ no

Which answer do you prefer?
☒ A ☐ B ☐ tie ☐ neither

Useful span from Answer A
The Cleveland Show came into existence as a spin-off of Family Guy. Seth MacFarlane initially conceived the series in 2007 while he was working on Family Guy and American Dad! for Fox. [...] The series centered on Cleveland Brown after he moved into a

Useful span from Answer B
Copy the smallest useful text span, or leave blank.
Press ⌘+Enter to apply

Optional notes
Uncertainty, factual concerns, or why neither answer is satisfactory.

Évaluation. Human annotation

But : comparer HiRAG classique à nos 3 variantes

Variante	Win rate	Densité span	Gain
HiRAG classique	-	28.7%	-
Naive	65.0%	32.0%	+0.30
Minmax budgeted	45.0%	32.1%	-0.10
MCTS	52.5%	30.4%	+0.05

Interface d'annotation

Packet 1 | Query mix:11

How did The Cleveland Show come into existence, and what was its premise?

Answer A

Development

The Cleveland Show came into existence as a spin-off of *Family Guy*. Seth MacFarlane initially conceived the series in 2007 while he was working on *Family Guy* and *American Dad!* for Fox. The show was developed around Cleveland Brown, a character already established on *Family Guy*, and Fox picked up the

Answer B

Creation and development

The Cleveland Show came into existence as a spin-off of *Family Guy*. According to the data, it was created by Seth MacFarlane, Richard Appel, and Mike Henry for Fox Broadcasting Company.

Your annotation

Does Answer A answer the question?

☒ yes ☐ partially ☐ no

Does Answer B answer the question?

☒ yes ☐ partially ☐ no

Which answer do you prefer?

☒ A ☐ B ☐ tie ☐ neither

Useful span from Answer A

The Cleveland Show came into existence as a spin-off of Family Guy. Seth MacFarlane initially conceived the series in 2007 while he was working on Family Guy and American Dad! for Fox. [...] The series centered on Cleveland Brown after he moved into a

Useful span from Answer B

Copy the smallest useful text span, or leave blank.

Press ⌘+Enter to apply

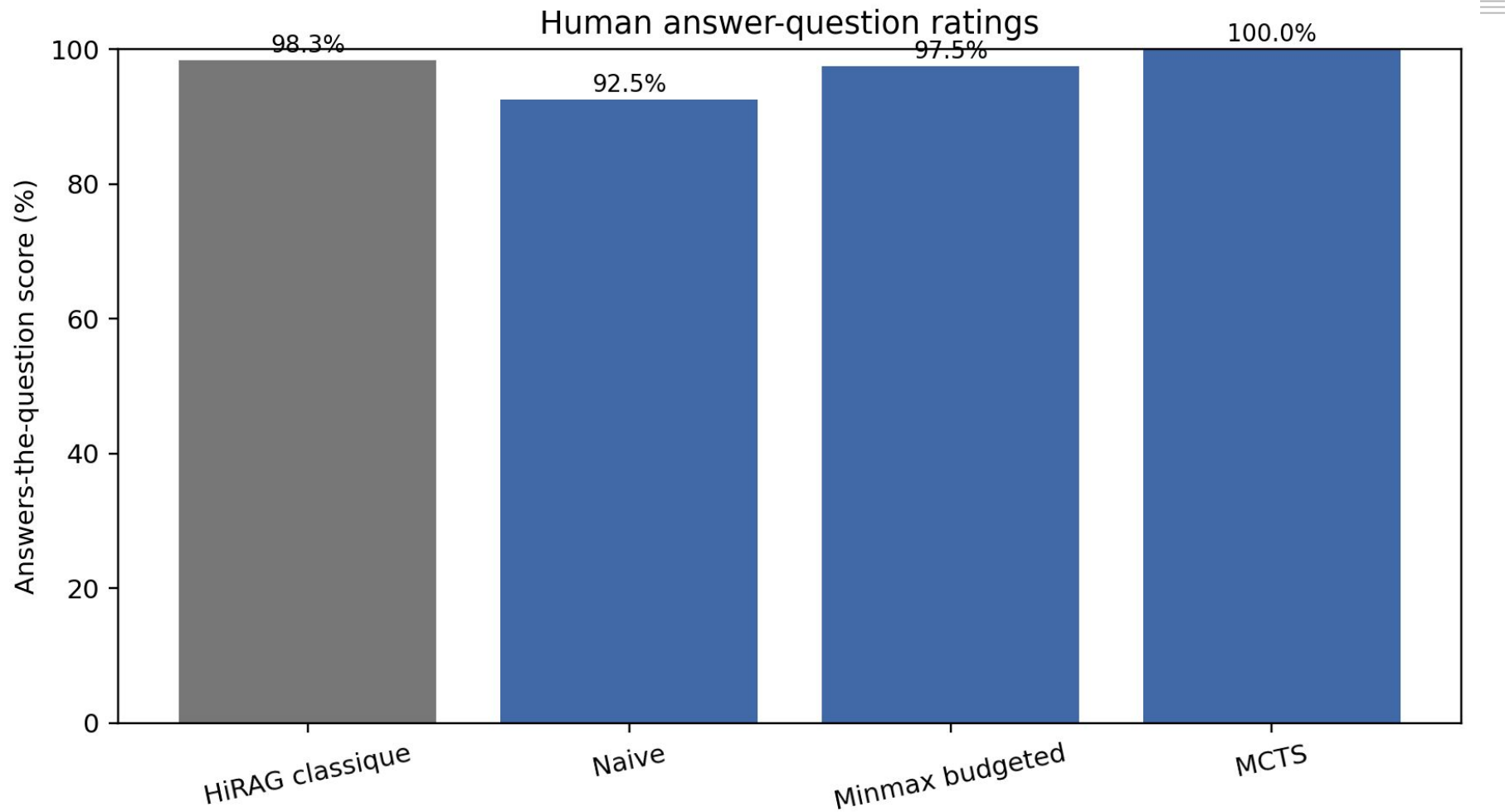
Optional notes

Uncertainty, factual concerns, or why neither answer is satisfactory.

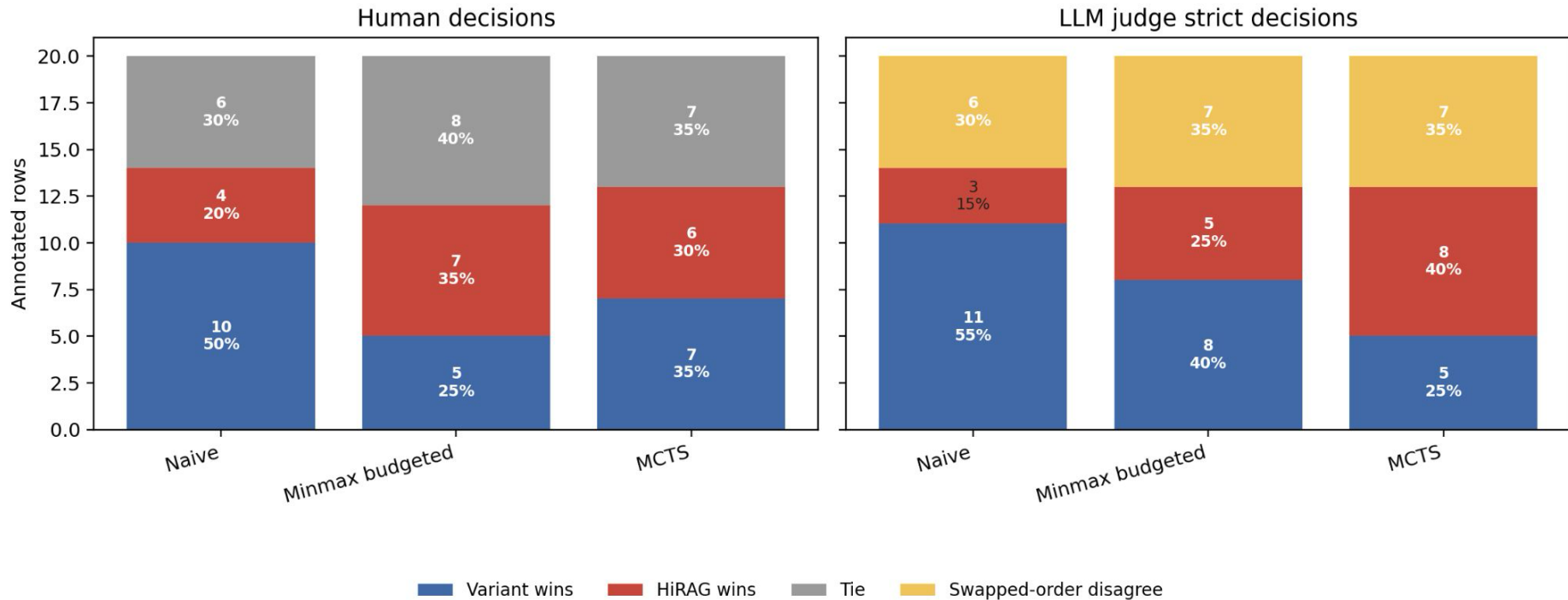
Évaluation. Human annotation

But : comparer HiRAG classique à nos 3 variantes (nouvelles données incluses) :

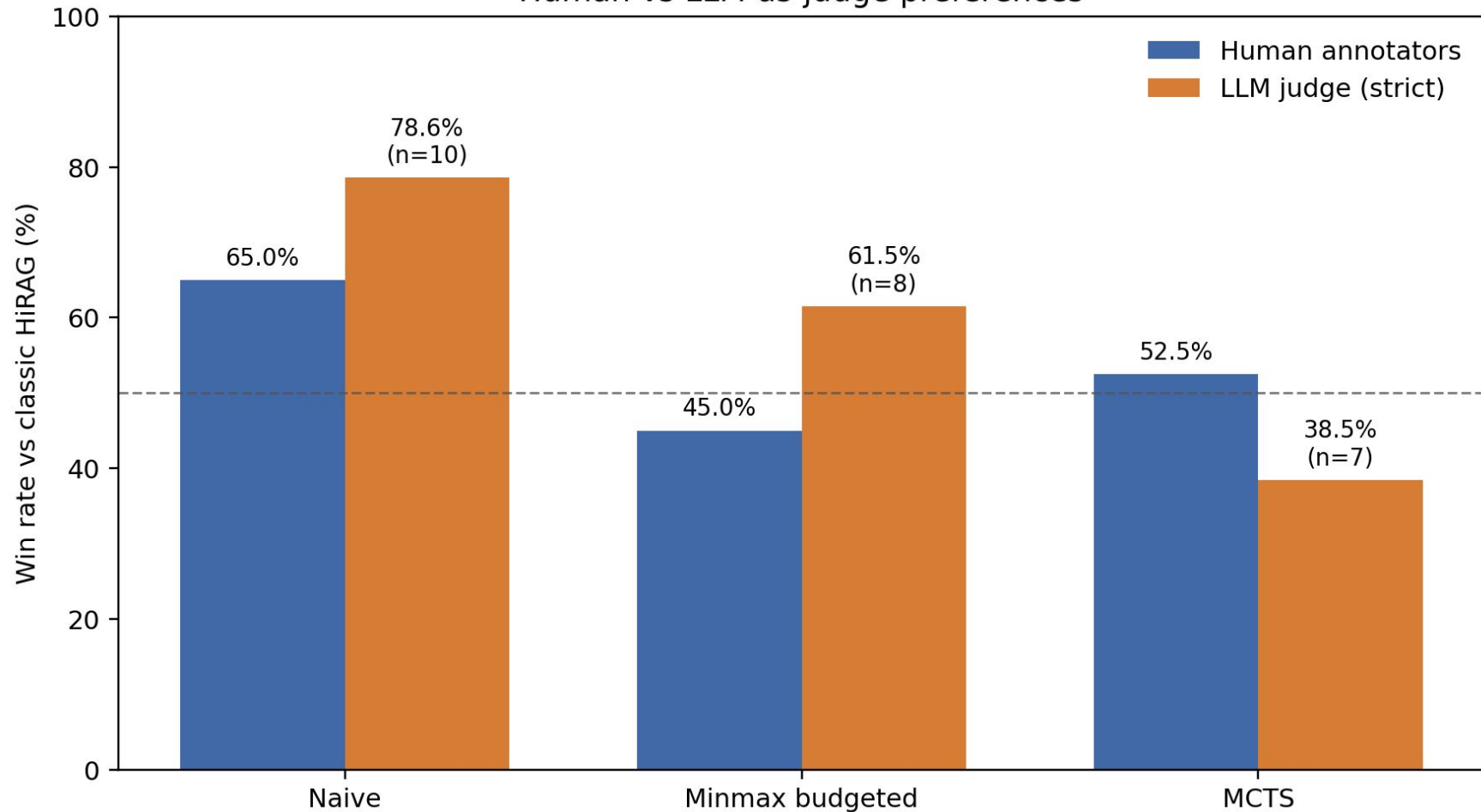
Variante	Human win rate vs HiRAG	LLM win rate vs HiRAG	Human gain	Span density
Naive	65.0%	78.6%	+0.30	32.0%
Minmax budgeted	45.0%	61.5%	-0.10	32.1%
MCTS	52.5%	38.5%	+0.05	30.4%



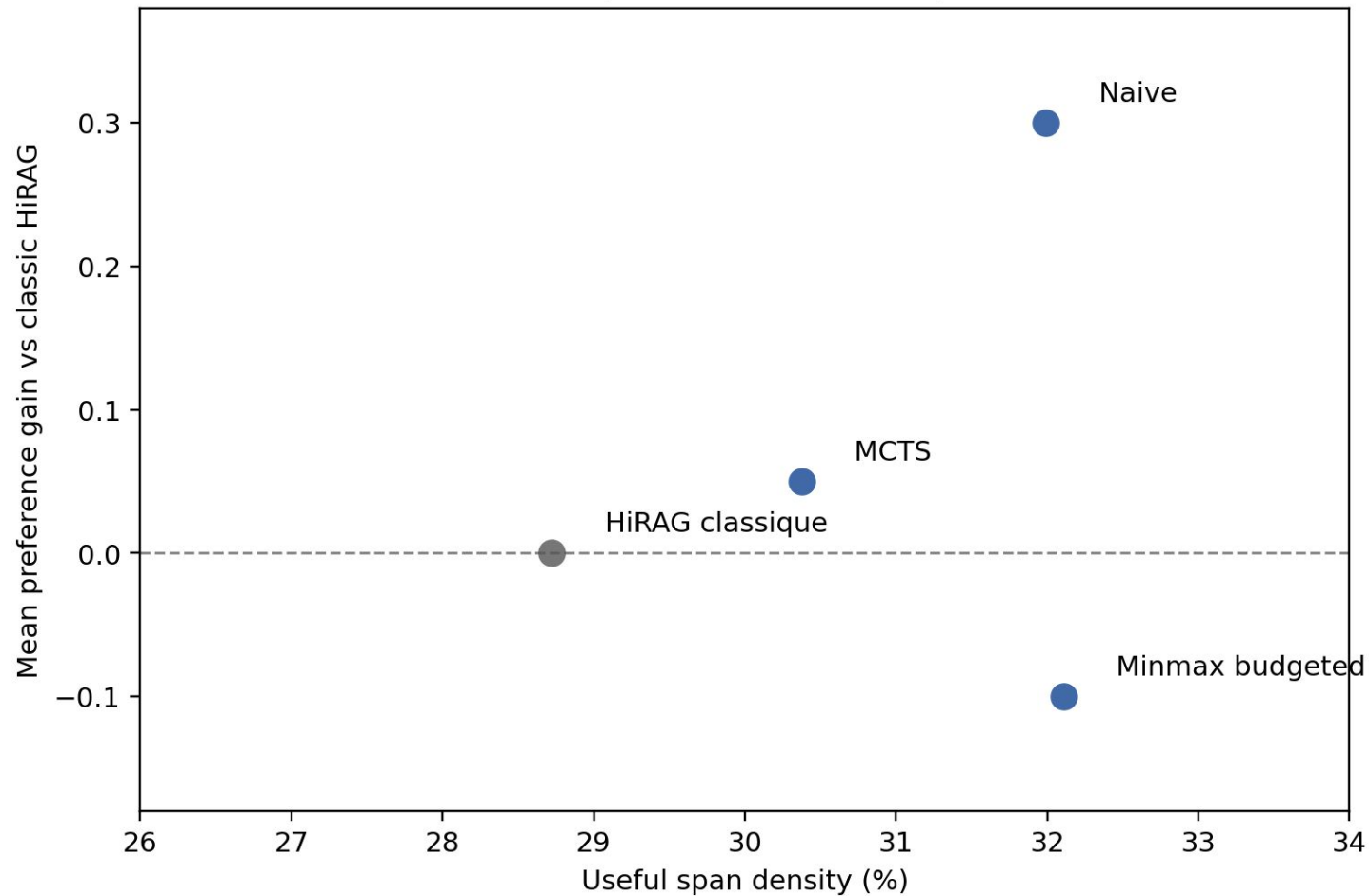
Decision breakdown on the same human-annotated pairs



Human vs LLM-as-judge preferences



Evidence compactness vs human preference



Conclusion

